



Institute of Information Technology
Jahangirnagar University, Savar, Dhaka-1342

Lab Report - 04

Lab Title: Unix — File Permission / Access Mode

Course Code: ICT-3102

Course Title: Operating System Lab

Submitted by:

Md. Shaon Khan

Id: 1984

Batch: 52 Batch

Session: 2022-2023

Supervised by:

Mehrin Anannya

**Assistant Professor, Institute of Information Technology
Jahangirnagar University**

Date: 12 May, 2026

1. Objective

The objective of this lab session is to develop a hands-on understanding of Linux file system operations and Unix-based file permission management. Specifically, the lab aims to:

- Create and navigate directories and files using terminal commands.
- Understand and apply wildcard metacharacters for pattern-based file listing.
- Create and edit text files using the vi editor and cat redirection.
- Analyze file contents using cat, cat -b, more, and wc commands.
- Interpret file permission strings from ls -l output.
- Modify file access permissions using both symbolic and numeric (octal) modes of the chmod command.

2. Theory

Linux is a multi-user operating system that enforces strict file permission controls to ensure security and data integrity. Every file and directory in a Linux system has associated access permissions that govern who can read, write, or execute it.

File Permission Structure

When the ls -l command is used, each file is prefixed with a 10-character permission string, for example: -rw-rw-r--. The first character indicates the file type (- for regular file, d for directory). The next nine characters are divided into three groups of three: Owner (u), Group (g), and Others (o), each containing read (r), write (w), and execute (x) bits.

Wildcard Metacharacters

Wildcards are special characters used in shell commands to match filenames based on patterns. The asterisk (*) matches zero or more characters, allowing commands like ls *.txt to list all files with the .txt extension. The question mark (?) matches any single character.

The vi Text Editor

The vi editor is a standard Unix text editor that operates in two primary modes: Command Mode (default, for navigation and issuing commands) and Insert Mode (for typing text, activated by pressing i). To save and exit, the user presses Esc to return to command mode, then types :wq and presses Enter.

The chmod Command

The chmod (change mode) command is used to modify file permissions. It supports two modes:

- **Symbolic Mode:** Uses letters (u, g, o, a) with operators (+ to add, - to remove, = to set exactly).
Example: chmod g+x file adds execute permission to the group.
- **Numeric (Octal) Mode:** Uses a three-digit octal number where Read = 4, Write = 2, Execute = 1. The digits correspond to Owner, Group, and Others respectively. Example: chmod 755 file sets Owner to rwx (7), Group to r-x (5), and Others to r-x (5).

3. Commands Used

The following table summarizes all commands used in this lab session:

Command	Description
ls	Lists files and directories in the current location
ls -l	Displays detailed listing with permissions, owner, size, and timestamps
ls file1*.txt	Lists files matching the wildcard pattern (starting with file1, ending in .txt)
ls fi*.txt	Lists files starting with "fi" and ending in ".txt"
ls *.txt	Lists all files with the .txt extension
mkdir	Creates a new directory
cd	Changes the current directory; cd alone returns to home
touch	Creates empty files or updates file timestamps
cat > file	Creates a new file and writes standard input into it (end with Ctrl+D)
cat file	Displays the contents of a file on screen
cat -b file	Displays file contents with line numbers for non-blank lines
more file	Displays file content one screen at a time
wc file	Counts and displays lines, words, and characters in a file
vi file	Opens the vi text editor to create or modify a file
chmod (symbolic)	Changes file permissions using letters (u/g/o) and operators (+/-/=)
chmod (numeric)	Changes file permissions using octal numbers (r=4, w=2, x=1)

4. Procedure

4.1 Directory and File Creation

Created the OS and Lab directories, navigated into OS, then created multiple files using the touch command.

Command:

```
$ mkdir OS Lab
$ cd OS
$ touch file1.txt file12.txt file2.txt notes.txt file1.pdf
```

Output:

```
(No output, files created silently)
```

Observation: The touch command created all specified files in a single call. The mkdir command successfully created two separate directories, OS and Lab, visible in the subsequent ls listing.

4.2 Using Wildcard Metacharacters

Demonstrated wildcard patterns to filter file listings based on name patterns.

Command:

```
$ ls file1*.txt
$ ls fi*.txt
$ ls *.txt
```

Output:

```
file12.txt file1.txt
file12.txt file1.txt file2.txt
file12.txt file1.txt file2.txt notes.txt
```

Observation: Each pattern produced a different subset of files. The * metacharacter effectively matched zero or more characters, narrowing or widening the result based on the prefix pattern provided.

4.3 Creating Files with cat Redirection

Used the cat command with output redirection to create text files with content typed directly from the terminal.

Command:

```
$ cat > notes.txt
This is note text created by Shaon Khan.
(Ctrl+D to save)

$ cat > routine.txt
Sunday: OS Lab
Monday: Networking
(Ctrl+D to save)
```

Output:

```
$ ls *.txt
file12.txt file1.txt file2.txt notes.txt routine.txt
```

Observation: The cat > filename syntax redirected keyboard input into the specified file. Pressing Ctrl+D sent the EOF signal, finalizing the file. Both files appeared in the subsequent directory listing.

4.4 Creating and Editing Files with vi

Used the vi editor to create hello.txt and enter content in Insert Mode, then saved and exited using :wq.

Command:

```
$ vi hello.txt
(Press i to enter INSERT mode)
This is file created by Shaon.
I am from IIT, JU.
(Press Esc)
:wq
```

Observation: The vi editor opened in Command Mode. Pressing i switched to INSERT mode (shown at the bottom of the screen). Text was typed successfully. After pressing Esc, the :wq command saved the file and returned to the terminal prompt.

4.5 Viewing and Analyzing File Contents

Used cat, cat -b, wc, and more to display and analyze the content of hello.txt and Hello1.txt.

Command:

```
$ cat hello.txt
$ cat -b hello.txt
$ wc hello.txt
$ more Hello1.txt
```

Output:

```
This is file created by Shaon.
I am from IIT, JU.

 1 This is file created by Shaon.
 2 I am from IIT, JU.

3 11 51 hello.txt
```

Observation: The cat command printed the raw file contents. cat -b added sequential line numbers to each non-blank line. The wc command revealed the file has 3 lines, 11 words, and 51 characters. The more command displayed Hello1.txt content one screen at a time.

4.6 Viewing Detailed File Listing

Used ls -l to inspect file permissions, ownership, and metadata for all files in the OS directory.

Command:

```
$ ls -l
$ ls -l hello.txt
```

Output:

```
-rw-rw-r-- 1 shaonkhan shaonkhan  0 May  6 15:58 file12.txt
-rw-rw-r-- 1 shaonkhan shaonkhan  0 May  6 15:58 file1.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
...
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
```

Observation: The ls -l output clearly displayed the 9-character permission string, the number of hard links, the owner name, the group name, the file size in bytes, and the last modification timestamp for every file. The hello.txt file showed 51 bytes, consistent with the wc output.

4.7 Modifying Permissions Using Symbolic Mode

Applied a series of symbolic chmod operations to hello.txt, verifying each change with ls -l.

Command:

```
$ chmod g+x hello.txt → -rw-rwxr--
$ chmod o+wx hello.txt → -rw-rwxrwx
$ chmod u+x hello.txt → -rwxrwxrwx
$ chmod u-rwx hello.txt → ----rwxrwx
$ chmod g-x hello.txt → ----rw-rwx
$ chmod o-r hello.txt → ----rw--wx
$ chmod o=x hello.txt → ----rw---x
$ chmod g=x hello.txt → -----x--x
```

```
$ chmod u=r hello.txt → -r---x--x
```

Observation: Each symbolic operation precisely modified the intended permission bits without affecting the others. The = operator was particularly notable in that it completely replaced the target group's permissions rather than adding or removing individual bits.

4.8 Modifying Permissions Using Numeric Mode

Applied numeric (octal) chmod values to hello.txt and verified the resulting permission strings.

Command:

```
$ chmod 444 hello.txt → -r--r--r--
$ chmod 000 hello.txt → -----
$ chmod 777 hello.txt → -rwxrwxrwx
$ chmod 562 hello.txt → -r-xrw--w-
```

Observation: The numeric mode provided a concise and efficient way to set all nine permission bits in a single command. chmod 000 removed all permissions entirely, while chmod 777 granted full access to all three groups. The value 562 decomposed to r-x (5) for Owner, rw- (6) for Group, and -w- (2) for Others, which was verified in the ls -l output.

5. Discussion

This lab session provided comprehensive, practical exposure to Linux file management and permission control. Several important observations were made during the session:

- Wildcard metacharacters proved to be highly efficient for filtering large directory listings. The asterisk (*) is especially powerful as it can match any string, making pattern-based file operations fast and convenient.
- The vi editor, while initially non-intuitive due to its modal design, offers powerful editing capabilities once the distinction between Command Mode and Insert Mode is understood. The :wq convention is a universal save-and-quit mechanism.
- The ls -l command is indispensable for diagnosing permission-related issues. The 10-character permission string encodes rich information about file type, ownership model, and access rights in a compact format.
- Symbolic chmod is more readable and human-friendly, making it ideal for targeted permission changes such as adding execute permission for a specific group. Numeric chmod is faster for setting complete permission profiles from scratch.
- The chmod 000 operation demonstrated that even the file owner can be locked out of their own file, highlighting the importance of careful permission management in multi-user environments.
- The wc command output (3 11 51 hello.txt) confirmed that the file had 3 lines (including a trailing newline), 11 words, and 51 bytes, providing a precise characterization of file content.

6. Conclusion

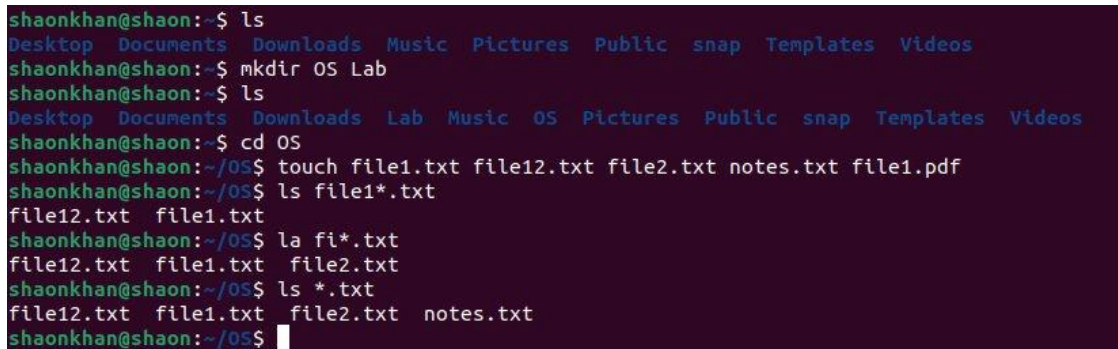
This laboratory exercise successfully achieved its intended objectives. Through a structured series of experiments, the fundamental aspects of Linux file operations were mastered, ranging from directory navigation and file creation to content analysis and permission management.

The ability to control file access through `chmod` is a cornerstone of Linux system administration and software development. Understanding both symbolic and numeric modes provides the flexibility to handle any permission-related task efficiently. The concepts practiced in this lab — particularly around the `chmod` command, `ls -l` interpretation, and `vi` editing — form the foundation for more advanced topics such as shell scripting, process management, and system security hardening.

Future lab sessions will build upon these skills, extending into areas such as process control, piping, I/O redirection, and network administration.

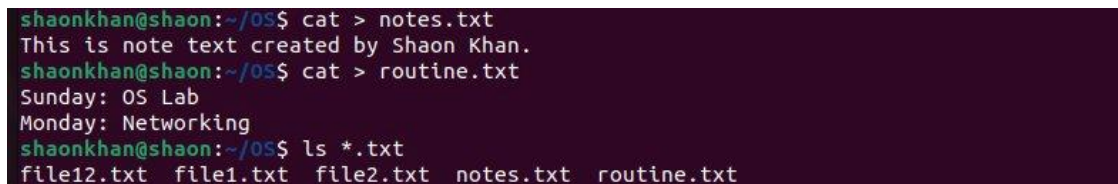
7. Lab Screenshots

The following terminal screenshots document all commands executed during this lab session in chronological order.



```
shaonkhan@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
shaonkhan@shaon:~$ mkdir OS Lab
shaonkhan@shaon:~$ ls
Desktop  Documents  Downloads  Lab  Music  OS  Pictures  Public  snap  Templates  Videos
shaonkhan@shaon:~$ cd OS
shaonkhan@shaon:~/OS$ touch file1.txt file12.txt file2.txt notes.txt file1.pdf
shaonkhan@shaon:~/OS$ ls file1*.txt
file12.txt  file1.txt
shaonkhan@shaon:~/OS$ la fi*.txt
file12.txt  file1.txt  file2.txt
shaonkhan@shaon:~/OS$ ls *.txt
file12.txt  file1.txt  file2.txt  notes.txt
shaonkhan@shaon:~/OS$
```

Figure 1: Creating directories, navigating filesystem, and using wildcard metacharacters (`ls file1*.txt`, `ls fi*.txt`, `ls *.txt`)



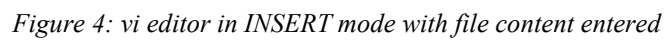
```
shaonkhan@shaon:~/OS$ cat > notes.txt
This is note text created by Shaon Khan.
shaonkhan@shaon:~/OS$ cat > routine.txt
Sunday: OS Lab
Monday: Networking
shaonkhan@shaon:~/OS$ ls *.txt
file12.txt  file1.txt  file2.txt  notes.txt  routine.txt
```

Figure 2: Creating files using `cat` redirection (`cat > notes.txt`, `cat > routine.txt`) and listing all `.txt` files



```
shaonkhan@shaon:~/OS$ vi hello.txt
```

Figure 3: Opening the `vi` text editor to create `hello.txt`




```

shaonkhan@shaon:~/OS$ ls -l
total 16
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file12.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file1.pdf
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file1.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file2.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 41 May 6 16:00 notes.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 37 May 12 18:26 practice_2.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 34 May 6 16:01 routine.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt

```

Figure 7: Detailed directory listing (`ls -l`) showing file permissions, ownership, size, and timestamps

```

shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod g+x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rwxr-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod o+wx hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod u+x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rwxrwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod u-rwx hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
----rwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod g-x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
----rw-rwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod o-r hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
----rw--wx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod o=x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
----rw---x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod g=x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-----x--x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod u=r hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-r-----x--x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$

```

Figure 8: Changing file permissions using symbolic mode `chmod` (`g+x`, `o+wx`, `u+x`, `u-rwx`, `g-x`, `o-r`, `o=x`, `g=x`, `u=r`)

```
shaonkhan@shaon:~/05$ ls -l hello.txt
-r---x--x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 444 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
-r--r--r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 000 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
----- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 777 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
-rwxrwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 562 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
-r-xrw--w- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
```

Figure 9: Changing file permissions using numeric (octal) mode chmod (444, 000, 777, 562)

```
shaonkhan@shaon:~/05$ cd
shaonkhan@shaon:~$
```

Figure 10: Returning to home directory using cd command

```
shaonkhan@shaon:~$ cat > Hello1.txt
Hello World!!
shaonkhan@shaon:~$ more Hello1.txt
Hello World!!
```

Figure 11: Creating Hello1.txt using cat redirection and viewing with more command